

Roteiro de Aula: Transformação I: Limpeza de Dados e Normalização

Guias Gerais

- Revisão do fluxo de dados (pipeline)
- Reforço das transformações vistas até o momento
- Apresentação de Feature Engineering
- Discretização de dados
- Técnicas de redução de dimensionalidade
- Aplicações práticas em IA

Referências

Revisão

Um framework para a Transformação de Dados (pré-processamento)

Limpeza -> Manipulação -> Redução

- Limpeza:
 - Tratar dados faltantes
 - Reduzir ruído
 - Eliminar duplicação
- Manipulação:
 - Normalizar
 - Discretizar
 - Criar atributos (*feature engineering*)
- Redução:
 - Redução de dimensionalidade
 - Redução de volume
 - Equilíbrio

Limpeza de Dados

É comum ouvir que, em projetos de ciência de dados, a maior fatia de tempo é dedicada ao processo de preparação dos dados. Tratando especialmente de casos em que não temos um conjunto de dados pré-estruturado para análise, é comum encontrarmos dados em variados formatos, com valores faltantes e não padronizados.

A limpeza de dados consiste no tratamento de casos desse tipo para permitir o processamento posterior destes. É função do cientista de dados compreender a origem dos dados, identificar as necessidades de ajuste e escolher a ação apropriada para cada caso.

Nossas tratativas serão concentradas no uso da biblioteca **pandas** para implementar esse tratamento. Isso não significa que este é o único método, porém a construção de *pipelines* de limpeza de dados através de uma linguagem de programação traz muita flexibilidade ao processo.

Importante: é essencial que o processo de limpeza de dados seja acompanhado por uma camada de auditoria. Essa camada registra todos os procedimentos adotados, provendo transparência ao usuário final. Por exemplo, se o processo de limpeza substitui dados faltantes por um valor *default*, a substituição deve ser relatada para que o usuário entenda as implicações disso nas análises fim.

Algumas Considerações

Para uma análise de dados eficiente, é importante que os dados apresentem algumas características:

1. Não ambiguidade: as representações escolhidas devem remover a ambiguidade na interpretação. Por exemplo, um registro de nascimento em Bom Jesus pode se referir a cidade no RS, no RN ou em SC. Nesse caso, a utilização da coluna **Estado** é essencial para desambiguar a informação;
2. Consistência: um dado não deve possuir múltiplas representações distintas em um dataset. Se temos os valores **M** e **Masculino** para se referir ao mesmo gênero de uma pessoa, devemos padronizar a representação e substituir os valores divergentes;
3. Completude: os dados devem ser completos em termos **individuais e agregados**. No primeiro caso, cada registro deve possuir valores válidos para todos os atributos necessários para a análise. Para o segundo, devemos garantir que registros individuais não foram perdidos durante o processamento.

Valores Faltantes (Tratamento de Nulos)

Iniciamos pelo caso mais comum: o que fazer se algum registro não inclui os dados em uma determinada coluna. Por exemplo, ao analisar uma planilha com as notas da turma, o coordenador percebe que um estudante não possui nota em uma determinada Unidade Curricular. Qual ação deve ser tomada?

É essencial que esses valores sejam identificados, quando existirem em dados importantes para a análise. No exemplo das notas, se o estudante for incluído em um cálculo de média da turma para as UCs, o valor resultante será "puxado para baixo" se sua nota for considerada 0. Para esse caso, a melhor tratativa é excluir o registro do estudante da análise das UCs em que não obteve nota.

Tipos de dados Faltantes

Podemos classificar os dados faltantes em 3 categorias:

1. Deficiências estruturais dos dados
2. Ocorrências aleatórias
3. Causas específicas

Uma deficiência estrutural dos dados ocorre quando um componente essencial foi omitido dos dados coletados. Esse caso é usualmente mais simples de tratar. Se, por exemplo, vemos registros que não possuem o nome do autor da Emenda no nosso dataset, podemos inferir que a emenda não foi identificada. Outro exemplo é um dataset que reflete características de imóveis vendidos na cidade, contendo um campo "varanda", preenchido com as categorias "aberta", "fechada", "integrada". Se o campo não está preenchido, podemos entender que esses imóveis não possuem varanda.

A falta de dados por causas específicas é mais comum quando analisamos cenários de coletas periódicas. Por exemplo, em um estudo clínico: se o paciente sair do grupo de estudos devido a um efeito colateral durante a realização do estudo, os dados finais não exibirão a análise completa deste

paciente. Este tipo é o mais difícil de corrigir, usualmente necessitando de apoio de especialistas da área para escolher o método mais apropriado.

Identificando Valores Faltantes

Existem algumas técnicas importantes para identificar valores faltantes. A primeira dela é contabilizar os registros faltantes para cada uma das nossas características (colunas). Podemos visualizar a distribuição e identificar as características com o maior número de valores faltantes para orientar a análise.

Utilizando pandas, nossa primeira tarefa é identificar se existem valores faltantes. Nessa biblioteca, valores numéricos não existentes são representados pelo valor **NaN**. Para dataframes que contenham valores de tipos complexos, o valor **None** também é compreendido como um valor faltante. A função `isna()` pode ser utilizada para identificar ambos os casos:

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_excel("bensimoveis-69cbc20141908.xlsx")
# Substituo String vazia por NA para incluir estes registros na contagem
# SE eu sei que existe um valor default (-1 por exemplo) para esses casos, posso
substituir por NA também
data = data.replace("", pd.NA)

# Contagem de valores faltantes, ordenada menor->maior
heatmap = data.isna().sum().sort_values()

# Podemos também gerar um sumário, calculando a porcentagem de registros faltantes
relativa = data.isna().mean() * 100
print(relativa.sort_values(ascending=False))

# Visualização da distribuição de valores faltantes
plot = heatmap.plot.bar(x=heatmap.index)
plt.show()
```

Essa análise inicial já nos dá uma base para a tomada de decisões. No dataset de exemplo, o campo **decomplemento** é o campo com maior número de valores faltantes. É comum um registro de imóvel não incluir um complemento do endereço?

A visualização individualizada por coluna nos dá uma compreensão inicial da situação do dataset. Informações adicionais podem ser extraídas quando analisamos a correlação entre as características dos dados. Por exemplo, no dataset de imóveis cedidos pelo governo, existe uma relação entre a cidade da cessão e algum valor faltante?

```
# número total de registros faltantes
por_municipio = data.isna().groupby(data['nmmunicipio']).sum()
# Porcentagem
relativa = data.isna().groupby(data['nmmunicipio']).mean() * 100

plt.figure(figsize=(12, 8))
```

```
plt.ticklabel_format(style='plain', axis='both')

# Filtrando apenas os 20 municípios com mais dados faltantes para não poluir o gráfico
top_missing = relativa.sum(axis=1).sort_values(ascending=False).head(20).index
ax = sns.heatmap(relativa.loc[top_missing], annot=True, cmap='YlOrRd')

plt.title("Deficiência de Coleta de Dados por Município (%)")
plt.show()
```

Temos três ações principais para os casos:

1. Excluir os registros com dados faltantes;
2. Substituir os registros por um valor computável;
3. Obter o valor de um dataset complementar

Para escolher o método apropriado, considere algumas questões:

1. Qual o objetivo da análise?
2. Tenho outras fontes de dados que podem ser utilizadas para substituir meu banco?
3. Existe um valor padrão apropriado para substituição?
4. É fácil obter novas amostras? Se eu tenho poucas amostras (linhas), ou a obtenção de novas amostras para substituir as removidas é difícil, ou impossível, podemos colocar prioridade na **manutenção** das amostras existentes, mesmo que hajam dados faltantes;

Para o caso em que precisamos remover todos os valores faltantes, o pandas provê uma função auxiliar **dropna**. É importante decidir em qual eixo ocorrerá a remoção: removemos a linha ou a coluna que contém dados faltantes? Quais as justificativas para essa escolha?

```
# Por padrão, remove linhas que contém NA em qualquer campo
data = data.dropna()

# Podemos remover apenas as linhas onde todos os valores são NA
data = data.dropna(how='all')

# Para remover linhas com valores faltantes em uma coluna específica, usamos
subset
data = data.dropna(subset=['vlareaconstruida'])

# Para remover as colunas, passamos o eixo explicitamente
data = data.dropna(axis="columns")

# Se queremos remover os registros que contém N ou mais valores faltantes, usamos
thresh
data = data.dropna(thresh=2)
```

Quais as consequências de remover registros do nosso dataset? Quando essa escolha é apropriada?

Se, ao invés de remover os registros com itens faltantes, desejamos substituí-los com um valor default, a biblioteca também provê esses mecanismos:

```
# Preenche todos os valores faltantes com o default 0
df_notas = df_notas.fillna(0)

# Preenche de acordo com o valor de cada coluna
df_notas = df_notas.fillna({'Nota':0, 'Avaliacao':'?'})

# Para um preenchimento mais complexo, baseado em interpolação, usamos fillna
# Podemos preencher com o último valor válido antes do NaN
df_notas = df_notas.fillna(method='ffill')
# Ou com o valor seguinte ao NaN
df_notas = df_notas.fillna(method='bfill')

# Substitui NaN pela média dos valores (sem contabilizar os NaNs)
df_notas = df_notas.fillna(df_notas.mean())
```

Para dados categóricos, a falta de uma informação pode ser representada por uma categoria específica. Por exemplo, se não tenho o nome do autor de uma emenda parlamentar, posso utilizar a categoria "Não Identificado" para codificar estes valores. Um cuidado deve ser tomado quando utilizamos categorias para substituir valores numéricos faltantes: uma substituição mal pensada pode gerar distorções em cálculos e estatísticas computadas após a substituição.

Um caso mais complexo se apresenta quando devemos substituir os dados por observações válidas, vindas de outro dataset. Nesse caso, podemos realizar uma junção dos datasets, considerando uma chave compartilhada entre eles.

```
df_notas = pd.read_csv("notas.csv")
df_notas_recuperacao = pd.read_csv("notas_recuperacao.csv")

# Mescla os dataframes, utilizando as colunas compartilhadas como chave
# (matricula, nesse caso)
df_notas_completas = pd.merge(df_notas, df_notas_recuperacao)
# Para deixar a condição do merge explícita, usamos o parametro on
df_notas_completas = pd.merge(df_notas, df_notas_recuperacao, on='matricula')
```

O método `merge` do pandas implementa funcionalidades conhecidas dos JOINS de SGBD. Em sua forma padrão, o método executa um *inner join* dos datasets, retornando a intersecção de ambos: todas as linhas onde as chaves existem nos 2 datasets.

Podemos modificar o método de unificação pelo parâmetro `how`

```
# Todas as chaves do primeiro dataframe
df_notas_completas = pd.merge(df_notas, df_notas_recuperacao, on='matricula',
how='left')
```

```
# Todas as chaves do segundo dataframe
df_notas_completas = pd.merge(df_notas, df_notas_recuperacao, on='matricula',
                               how='right')
# União de todas as chaves de ambos os frames
df_notas_completas = pd.merge(df_notas, df_notas_recuperacao, on='matricula',
                               how='outer')
```

TODO: Gerar diagrama de inner, outer, left e right joins

É essencial que o método utilizado para "imputar" os valores faltantes seja registrado em trilhas de auditoria e documentado em código fonte. Esse registro auxiliará a explicar os resultados obtidos para o cliente final, bem como permitirá repetir a operação caso novas amostras dos dados sejam coletadas.

As técnicas demonstradas acima consideram medições estatísticas para preencher os valores faltantes, porém não são as únicas. Existem maneiras interessantes de gerar estes valores através de procedimentos de aprendizagem de máquina. Busquem informações sobre os seguintes métodos:

- KNN (K vizinhos mais próximos)
- Regressão Linear
- Regressão Logística
- Tree Ensembles

Sobre Auditoria

É muito importante que o processo de transformação de dados seja auditável. Isso permite que discrepâncias ou resultados inesperados sejam investigados pela equipe de analistas. Uma auditoria desse processo pode conter:

1. Registro de erros: um registro de erros, em formato de tabela ou arquivo, armazena informações sobre erros durante a extração e transformação dos dados.
2. Auditoria de transformações: registrar os passos de transformação, as regras adotadas e os metadados do dataset.

Utilizando as funções descritivas dos dataframes podemos obter informações úteis para nossas transformações:

```
#Quantidade de linhas do dataframe
tam_orig = len(df)

df = df.dropna()
# Quantidade após a transformação
tam_novo = len(df)

print(f"{tam_orig - tam_novo} linhas com valor nulo removidas!")
```

Se o número de linhas com erro for muito alto, isso pode indicar um problema sério com a fonte dos dados. Em casos extremos, isso pode invalidar a construção de análises apropriadas.

Para que as informações sobre a operação sejam persistidas, é importante que nosso processo permita a persistência da auditoria. Isso pode ser feito por meio de bibliotecas de *log*, através de uma tabela de auditoria no banco de dados, ou através de um dataframe auxiliar:

```
df_auditoria = pd.DataFrame([], columns=['Origem', 'NLinhas', 'Info', ''])
```

Com o avanço das tecnologias de armazenamento, o "espaço em disco" tornou-se uma comodidade barata, em comparação com o custo de processamento. Isso deu origem a serviços como os DataLakes, que armazenam todo tipo de dados, e não só os dados prontos para consumo.

Nesse contexto, é comum que pipelines de análise de dados gerem versões intermediárias dos datasets, armazenando-os como *backups*. As versões intermediárias podem provar-se essenciais para o diagnóstico e correção de erros no pipeline de processamento dos dados.

Normalização

Algumas tarefas na análise de dados exigem que trabalhem com dados **normalizados**. Destas tarefas, algumas classes de normalização são importantes:

Normalização da estrutura

Antes mesmo de iniciar a normalização dos dados, é importante analisar se a estrutura do dataset é a mais adequada para nossos objetivos. No geral, esperamos que os dados estejam em um formato tabular, contendo um cabeçalho informativo e um conjunto de linhas que representam *fatos* ou *observações* . Especialmente, esperamos que cada tabela contenha o conjunto completo de *variáveis* (também chamadas *características*) refletido em colunas.

Muitas das regras de normalização de banco de dados são aplicáveis a esses dados. Uma exceção importante se dá nos relacionamentos: poucas ferramentas de análise de dados trabalham com relacionamentos no modelo de SGBDs, portanto, é comum que os *datasets* na análise de dados apresentem o resultado de um *join* . Um modelo interessante de normalização da estrutura é proposto pelo conceito de *Tidy Data* (dados arrumadinhos)([link do artigo](#)). Nessa proposta, a normalização dos dados é definida por 3 regras:

1. Cada coluna é formada por **1 variável**
2. Cada linha representa **1 observação** completa
3. Cada tabela apresenta **1 tipo de observação**

Podemos ilustrar esse conceito com um *dataset* de notas dos alunos de uma turma ficcional do curso de Ciência de Dados e IA. O *dataset* original é como segue:

0,15	0,20	0,15	0,10	0,20	0,20	nome	N1	N2	N3	N4	N5	N6	Pedro Ilustração	8.40	2.49	5.83
7.85	10.00	5.97	Virgínia Padrão	7.20	7.23	2.29	4.27	2.05	6.84	Fern Talvez	9.28	4.31	7.68	6.34	2.14	6.43
8.76	9.00	5.93	0.00	10.00	7.96	Caska Padrão	7.49	6.78	10.00	0.62	8.39	8.66	Edward Exemplo	8.82	7.81	10.00
0.34	4.75	10.00	José Molde	6.52	8.18	10.00	4.90	10.00	6.05							

Neste dataset, a primeira regra é imediatamente violada. De fato, temos algumas variáveis representadas na forma de linhas:

1. A primeira linha contém o **peso** de cada nota para a formação da média final.
2. A linha 2 contém estatísticas do dataset, nominalmente a média da turma para cada nota.

Uma análise apropriada desses dados precisa associar o peso de cada avaliação à nota obtida pelo aluno. Essa estrutura permite a associação do peso de cada nota na linha da pontuação do aluno?

Também podemos entender que as colunas N1, N2, N3, etc. representam uma mesma variável: a nota do aluno. Com essa percepção, vemos a violação da 2ª regra: as linhas não representam uma única observação (uma nota do aluno), representam várias.

Para normalizar esse dataset, de acordo com as regras do *Tidy Data*, devemos aplicar 3 transformações principais:

1. Transformar a linha de pesos de avaliação em 1 coluna;
2. Transformar as múltiplas colunas de notas em 2 colunas
 1. Uma coluna com a nota obtida pelo aluno
 2. Uma coluna com o "nome" da nota (N1, N2, etc.)
3. Associar as novas colunas ao estudante

O processo de transformar colunas em linhas é denominado *melt*. Podemos aplicar este conceito no pandas:

```
# Usamos header=None para que a primeira linha não seja considerada como cabeçalho
df = pd.read_csv("bd1.csv", header=None)

# Primeira linha tem os pesos de cada prova
pesos = df.iloc[0][1:]
# Ids: Na 2ª linha, a partir da 2ª coluna
ids = df.iloc[1][1:]
# As notas aparecem a partir da 3ª linha
df_notas = df.iloc[2:]
df_notas.columns = df.iloc[1]
df_notas = df_notas.rename(columns={"Coluna 1": "nome"})

# zip é um iterador que percorre 2 listas
# Usamos dict para gerar um dicionário onde as chaves são as notas e os valores
# são os pesos
dict_notas_pesos = dict(zip(ids, pesos))
# Transforma um dataframe "largo" em um dataframe "longo"
# Um DF largo apresenta uma linha por entidade e variáveis nas colunas (Uma tabela
# é um DF largo)
# No nosso caso, cada linha é um aluno e as colunas são suas notas
# Um DF longo pode conter multiplas linhas por entidade, cada uma representando
# uma observação específica.
# Após a conversão, cada linha representa uma nota específica do aluno, com seu
# peso associado
# https://pandas.pydata.org/docs/reference/api/pandas.melt.html
df_melted = df_notas.melt(
    id_vars=["nome"],          # Variável ID - Nome do aluno
    value_vars=ids,            # Variáveis de valor - "nomes" das notas (N1, N2,
```



```

etc...)
    var_name="avaliacao", # Os ids das colunas serão valores da nova coluna
    'Avaliacao' (N1, N2, etc..)
    value_name="nota",    # Os valores das colunas, as notas em si, serão
    explodidos na coluna 'Nota'
)
# Agregamos o peso de cada avaliação ao dataframe final
df_melted["peso"] = df_melted["avaliacao"].map(dict_notas_pesos)

```

Com essa transformação, nosso dataframe final fica com a estrutura seguinte:

```

| nome | avaliacao | nota | peso | | Pedro Ilustração | N1 | 8.4 | 0,15 | | Virgínia Padrão | N1 | 7.2 | 0,15 | | Fern
Talvez | N1 | 9.28 | 0,15 | | Maria Talvez | N1 | 8.76 | 0,15 | ... | Fern Padrão | N2 | 7.49 | 0,20 | | Edward Exemplo |
N2 | 7.81 | 0,20 | | José Molde | N2 | 8.18 | 0,20 | | Stark Talvez | N2 | 7.68 | 0,20 |

```

O número de linhas aumenta, já que cada coluna de notas vira uma linha de observação. Mas este dataset é mais flexível para manipulações e ajustes. Por exemplo, se um determinado aluno não possui nota na avaliação N6, podemos facilmente excluir a linha dessa observação e o cálculo da média da turma na avaliação não será afetado. Se tentássemos fazer essa operação no dataset original, quais alternativas teríamos?

Normalização da Grafia

No caso de valores textuais, a grafia dos valores pode apresentar variações sutis que prejudicam a análise. Tivemos um exemplo disso no dataset de emendas parlamentares quando buscamos municípios com nomes diferentes no último exercício. Algum de vocês voltou no exercício anterior, onde agrupamos os dados agrupados por município, para refazer a análise após detectar a duplicação nos nomes?

Muito comum quando tratamos dados originados de inclusão humana, casos como erros de digitação, inclusão ou omissão de acento e nomenclaturas distintas (SC ou Santa Catarina, por exemplo) para um mesmo valor impactam o trabalho da análise de dados. Existem inúmeras maneiras de detectar esses casos.

Um primeiro passo, considerando um dataset de tamanho limitado, é listar os valores distintos ordenados alfabeticamente:

```

import pandas as pd

data = pd.read_csv("EmendasParlamentares.csv", encoding="iso-8859-1", sep=';')

print(data['UF'].sort_values().unique())

```

Caso tenhamos informações adicionais ao valor textual, podemos utilizá-las na desambiguação. Por exemplo, se temos um número de CPF, podemos detectar grafias distintas de um nome de pessoa agrupando por essa coluna. No caso das emendas parlamentares, temos o código do município para orientar nossa busca:

```

num_unique = data.groupby('Código Município IBGE')['Município'].nunique()
print(num_unique[num_unique>1])

```

Para normalizar valores textuais, temos algumas estratégias principais. Se eu sei qual é o valor mais apropriado, posso substituir as grafias incorretas pelo valor. No caso das emendas parlamentares, podemos fazer isso cruzando os dados com uma base de municípios extraída do IBGE

```
municipios = pd.read_excel('RELATORIO_DTB_BRASIL_2024_MUNICIPIOS.xls')
# As primeiras linhas não contém dados, mas descrição da tabela. Ignoramos
municipios=municipios.iloc[5:]
# A linha contém o cabeçalho, portanto, nossas colunas
municipios.columns=municipios.iloc[0]
municipios.head()
# Não preciso repetir o nome das colunas nos dados. Iniciamos na linha seguinte
municipios=municipios.iloc[1:]

# Ajusto os dados originais com os dados cruzados
data[data['Código Município IBGE' = municipios['Código Município Completo']]]
['Município'] =
# Para o merge, a coluna chave deve estar presente em ambos os dataframes
municipios = municipios.rename(columns={'Código Município Completo':'Código
Município IBGE'})
# Merge dos datasets. A partir daqui, posso usar Nome_Município nas análises
data = data.merge(municipios[['Código Município IBGE','Nome_Município']],
on='Código Município IBGE', how='left')
```

Uma normalização mais genérica pode ser feita através das definições de unicode. Podemos, por exemplo, remover as acentuações dos nomes de municípios com a normalização NKFD.

```
import unicodedata

def normalizacao_unicode(nome:str):
    if pd.isna(nome): return nome
    # Normaliza para a forma NFKD
https://www.otaviomiranda.com.br/2020/normalizacao-unicode-em-python/
    nfkd_form = unicodedata.normalize('NFKD', str(nome))
    return "".join([c for c in nfkd_form if not
unicodedata.combining(c)]).lower().strip()

data['Municipio_Normalizado'] = data['Município'].apply(normalizacao_unicode)
```

Devemos porém tomar um cuidado importante com essa normalização! Na normalização acima, temos a lista a seguir de municípios com grafia "duplicada":

aracoiaba [ARAÇOIABA, ARACOIABA]

arapua [ARAPUÃ, ARAPUÁ]

goiana [GOIANÁ, GOIANA]

ipira [IPIRÁ, IPIRA]

ipora [IPORÁ, IPORÃ]

marau [MARAU, MARAÚ]

parana [PARANÃ, PARANÁ]

quixaba [QUIXABÁ, QUIXABA]

Porém, se analisamos o código do município IBGE, Iporá e Iporã são 2 municípios diferentes! Essa normalização de dados é "destrutiva", no sentido de que não podemos restaurar o valor original após a normalização. Então é sempre importante tomar este cuidado com valores que representam classes mais complexas.

Outras operações características da normalização textual:

- Padronizar a capitalização (texto todo minúsculo, ou maiúsculo)
- Garantir que valores numéricos utilizem um separador decimal consistente (substituir , pelo . por exemplo)
- Remover espaços em branco no início/fim dos valores (`data['Município'].str.trim()`)
- Formatar representações de data: formato americano para internacional, transformar em timestamp, etc.

Normalização Numérica

Outro fator relevante da normalização na análise de dados é o ajuste nas escalas de valores numéricos. Este passo de processamento é especialmente relevante em dados preparados para modelos estatísticos ou de machine learning: se as características possuem magnitudes muito distintas, é possível que a característica com os maiores valores se sobreponha as demais.

No dataset de imóveis do poder legislativo temos uma situação desse tipo: aqui temos uma coluna referente a área do imóvel e uma coluna referente ao valor venal do mesmo. É natural que o valor monetário seja ordens de grandeza superior ao tamanho do imóvel. Se aplicamos esses dados em um algoritmo de machine learning que depende de uma noção de distância euclidiana $(x_1, y_1) - (x_2, y_2)$, a "distância" entre valores monetários será, usualmente, maior do que a distância entre dois pontos de área. Isso pode levar o algoritmo a entender que o valor monetário é mais importante do que a área, pois alterações nesse valor *movimentam mais* o ponteiro na tarefa de classificação.

Existem 2 métodos básicos de normalização. O primeiro é denominado *min-max*, e utiliza os extremos do dataset para normalizar os valores no intervalo [0, 1]. Considerando que x é o valor original, $\max(x)$ e $\min(x)$ são os valores com base em todos os valores do atributo, a fórmula da normalização é:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

A normalização *min-max* preserva a distribuição original dos valores, e define um intervalo regular para os dados (0 a 1).

Outra maneira comum de normalizar dados numéricos é com a padronização (*z-score*). Este método de normalização centraliza os dados, fazendo com os dados resultantes tenham **média = 0** e **desvio padrão = 1**.

Isso significa que o resultado é uma *distribuição normal* dos valores originais. Este método preserva a distribuição dos valores, computando o resultado da normalização com o uso da média e desvio padrão:

$$z = \frac{x - \mu}{\sigma}$$

Olhando para as fórmulas, quais são as maiores diferenças entre as duas metodologias?

| Característica | Min-Max | Z-score | | intervalo final | [0,1] | não limitado | | sensibilidade | Altamente sensível aos extremos | Menos sensível | | distribuição | Preserva a distribuição original | Transforma em uma distribuição normal | | principal uso | redes neurais, processamento de imagem | PCA, SVN, KNN |

Quando utilizar *z-score*:

- Os dados já seguem uma distribuição normal
- Serão usados em algoritmos que exigem distribuições normais (PCA)
- Preciso tomar cuidado com os extremos

Estes métodos de normalização são especialmente úteis quando precisamos aplicar técnicas de redução de dimensionalidade, como PCA, ou métodos de classificação baseados na descida de gradiente.

Uma forma conhecida de alterar a escala dos dados é aplicar a escala logarítmica. Nesse modelo, ao invés de analisar o valor original dos dados, trabalhamos com o logaritmo dos valores. Para a visualização de dados, a base 10 é usualmente preferida para simplificar os eixos (marcas 10, 100, 1000, etc..). Na estatística e em modelos de machine learning, é comum utilizar o logaritmo natural.

```
# Logaritmo natural
df['log_e'] = np.log(df['column_name']).
# Log base 10
df['log_10'] = np.log10(df['column_name']).
# Log base 2
df['log_2'] = np.log2(df['column_name']).
# Log natural +1 (Log é indefinido para 0, portanto a função é Log (x+1))
df['log_ep1'] = np.log1p(df['column_name'])
```

Mas qual a finalidade de aplicar a escala em log?

Em termos gerais, a escala em log é adequada para visualizar dados que abrangem um intervalo muito amplo de valores, ou onde o crescimento é exponencial. Por exemplo, ao mensurar o crescimento de colônia de bactérias, ou a expansão dos casos de COVID, o uso do log pode auxiliar na análise de tendências, deixando-as mais claras.

Uma escala logarítmica mostra a **taxa de crescimento** em relação aos demais datapoints (porcentagem). Se um eixo em escala linear representa quantas unidades foram adicionadas ao valor precedente, um eixo em escala logarítmica representa quantas vezes a quantidade posterior foi multiplicada.

Os conteúdos a seguir ilustram o quão útil é a escala logarítmica para a análise de tendências:

<https://www.lrs.org/2020/06/17/visualizing-data-the-logarithmic-scale/>

<https://data.europa.eu/apps/data-visualisation-guide/logarithmic-scales>

